



蚂蚁集团配置即代码 的规模化实践之路

演讲人：何子波

蚂蚁集团 / 高级技术专家



目录

01 浅谈配置管理

02 配置即代码

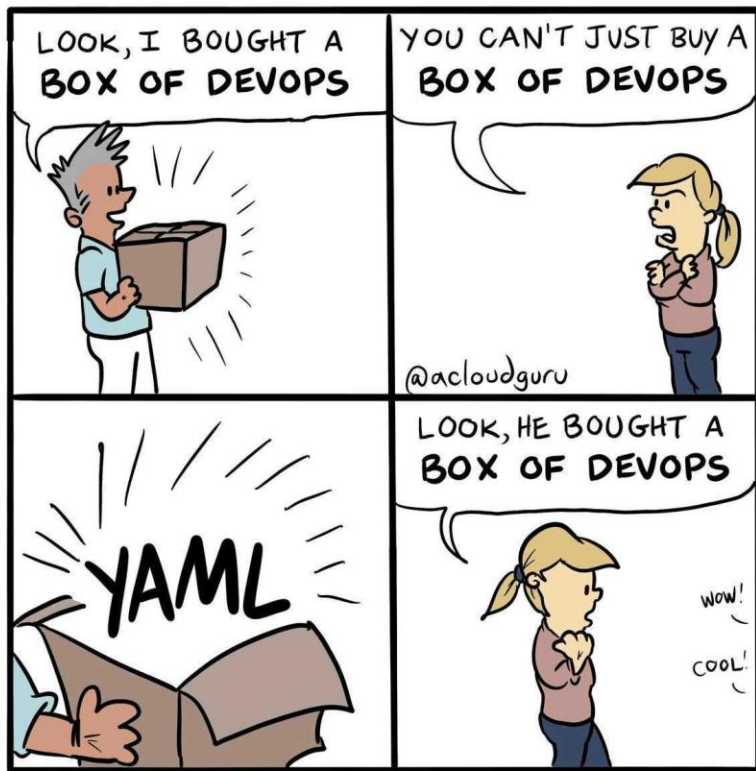
03 蚂蚁落地实践

04 总结与展望

01

浅谈配置管理

● 老生常谈的配置管理



传统的配置管理

分布式配置中心

✓ Kubernetes YAML Strikes

Kubernetes Configuration in 2024

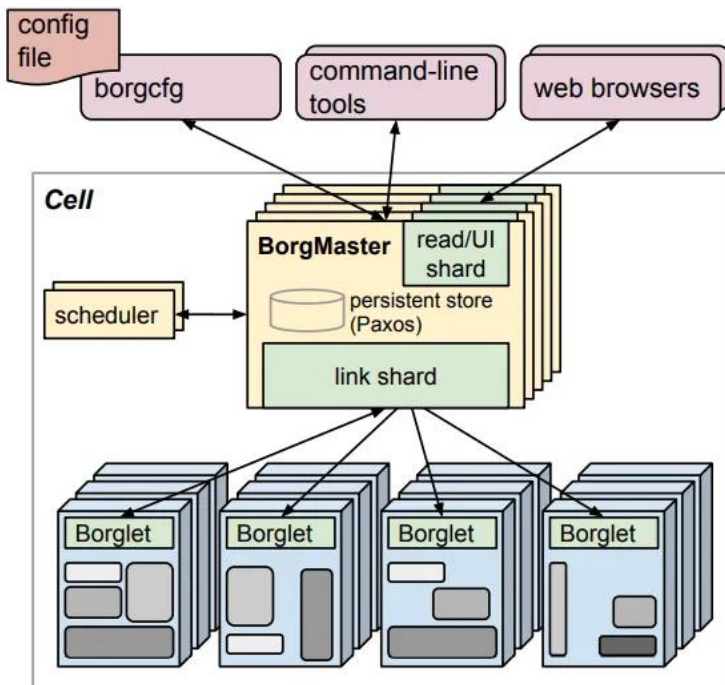


➤ 配置格式五花八门，YAML、配置领域专用语言(DSL)、通用编程语言(GPL)

➤ 各种配置工具层出不穷，热闹非凡，Helm、Kustomize 仍然最流行

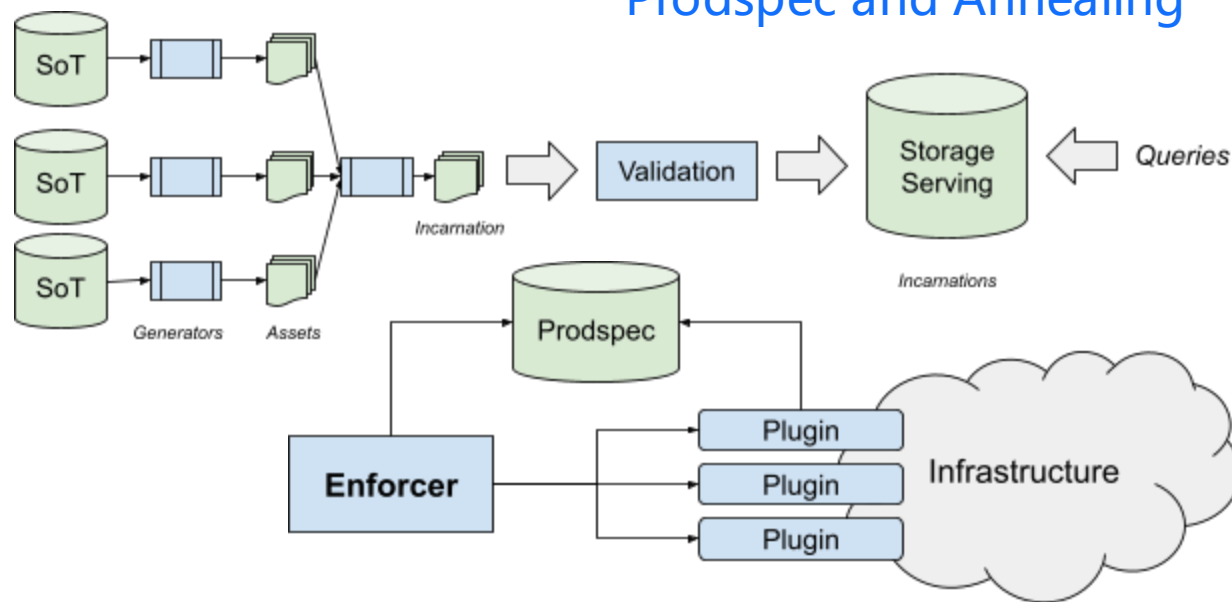
02 配置即代码

外观 – Google 的思路



```
job hello_world = {  
  runtime = { cell = 'ic' }           // Cell (cluster) to run in  
  binary = '../hello_world_webserver' // Program to run  
  args = { port = '%port%' }         // Command line parameters  
  requirements = {                   // Resource requirements (optional)  
    ram = 100M  
    disk = 100M  
    cpu = 0.1  
  }  
  replicas = 10000 // Number of tasks  
}
```

Prodspec and Annealing



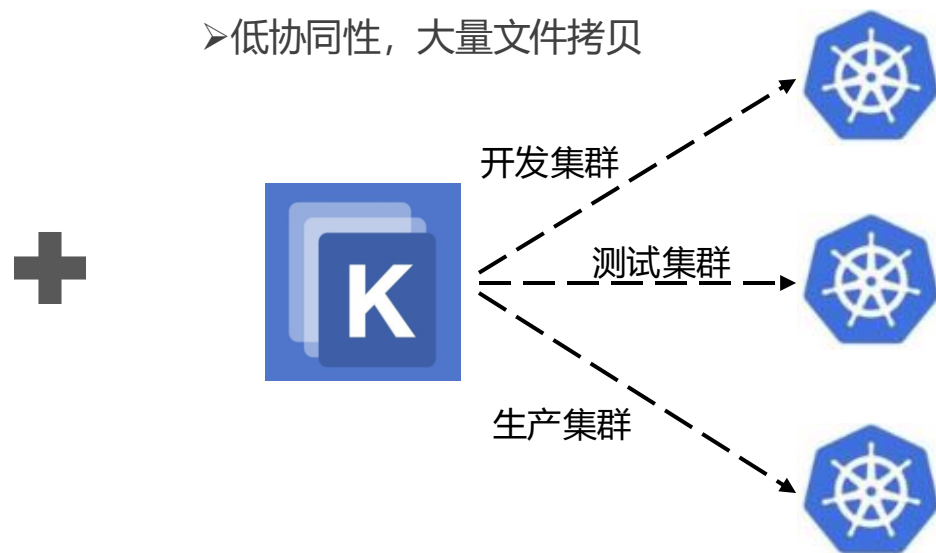
Borg接入三件套：BCL/borgcfg/WebConsole

内省 – 蚂蚁运维体系的困局

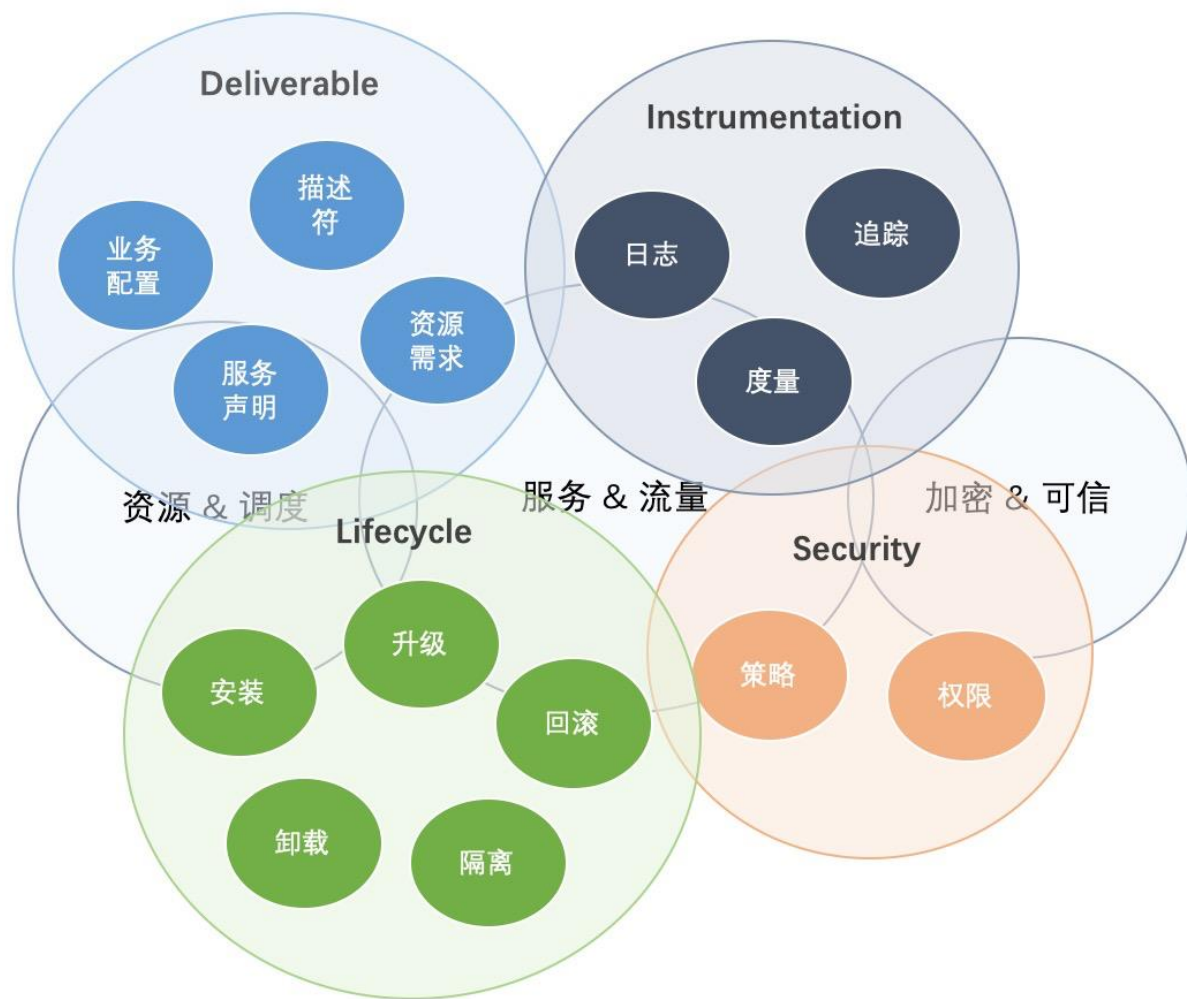


- 运维平台功能透明度不足，大量黑盒代码逻辑
- 运维平台高度定制化灵活性不足，无法满足应用个性化运维需求
- 资源瓶颈，无法复用和发挥 SRE 沉淀的运维经验

- 纯开源技术栈，工程化水位低
- 无规划化流程，缺少技术风险防控
- 低协同性，大量文件拷贝



语言化背后的思考



陷阱 1：没有把配置作为一种编程语言

如果你不是为之专门设计一种语言，那么最终用到的“语言”就不太可能是一种好语言。尽管配置语言描述的是数据而不是行为，但它们仍然具有编程语言的其他特征。



陷阱 2：设计特殊的语言功能

特殊的语言比正式设计的等效语言更复杂，并且通常具有低效的表达能力。与其祈祷配置系统不会复杂到需要一定的编程结构，不如在初始阶段就考虑到这些需求。



陷阱 3：使用现有的通用脚本语言 e.g. Python

使用通用脚本语言的实现方式往往是重量级的，并且/或则需要使用侵入式沙盒来确保密闭性。由于通用语言可以访问本地系统，处于安全考虑，可能还是需要沙盒。



配置存储方式的思考

Polyrepo (多仓库)

优势

- 配置跟着业务代码仓库走，开发者不需要感知额外的配置仓库，认知成本低
- 代码量和复杂性可控，不用担心可扩展问题
- 利于权限控制，权限分配粒度可以做非常细

劣势

- 难以建立统一的代码规范，平台团队缺少约束力
- 依赖管理与升级非常复杂，不利于 SRE 进行批量的配置变更

VS

Monorepo (单体仓库)

优势

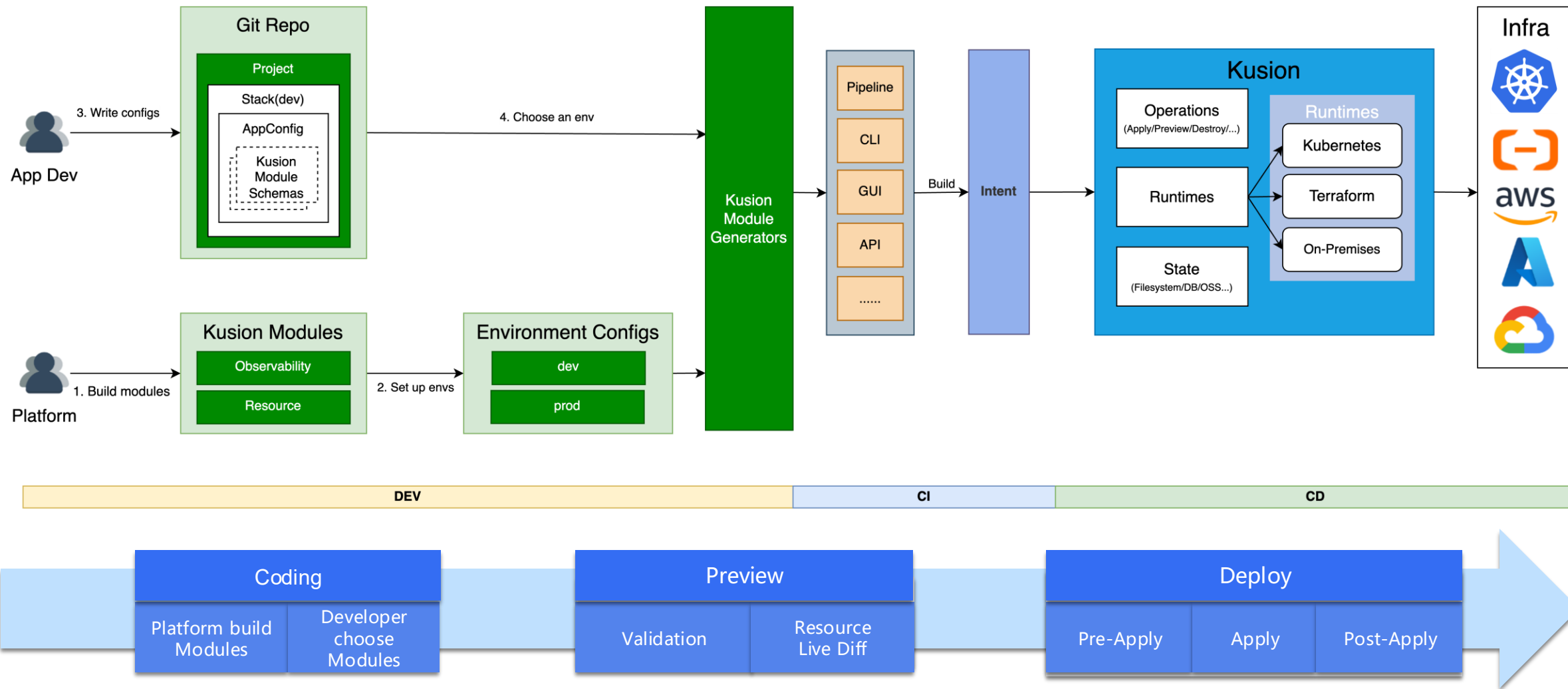
- 易于规范代码，中心化流水线更容易把控代码质量和风格，同时依赖管理与升级更加简单
- 利于与 SRE 进行批量配置变更与重构，且易于代码重用
- 利于构建中心化的配置自动化层，便于上游即成

劣势

- 开发者需要感知额外的配置仓库，有一定的学习成本
- 难以实现细粒度的权限控制，并且随着代码文件、数量越来越多，复杂性会越来越高，可能存在可扩展性问题



蚂蚁配置即代码整体思路



Kusion Configuration Language (KCL)

模型 & 集成



集成开发包



工具链

导入

格式化

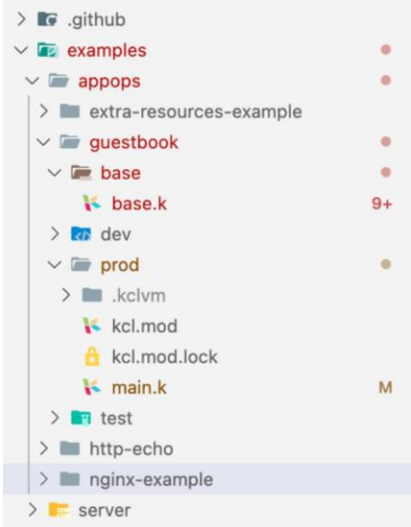
代码检查

测试

文档

注册中心

模块管理



```
examples > appops > guestbook > prod > main.k
You, 1 second ago | 3 authors (You and others)
1 import konfig.models.kube.frontend
2 import konfig.models.kube.frontend.container
3 import konfig.models.kube.frontend.service
4 import konfig.models.kube.templates.resource as res_tpl
5
6 # Application Configuration
7 guestbookFrontend: frontend.Server {
8   name = "frontend"
9   mainContainer = container.Main {
10     name = "php-redis"
11     env.GET_HOSTS_FROM: {
12       value = "dns"
13     }
14     ports = [{containerPort = 80}]
15   }
16 }
17
18 You, 1 second ago • Uncommitted changes
19
```

代码助手

高亮

诊断

跳转

格式化

补全

运行

语言服务器协议

KCL 语言服务器

annotations

configMaps

database

annotations: {str:str}

Annotations is an unstructured key value map stored

高性能

- LLVM 优化器
- Native + WASM Target
- 代码索引、增量编译

稳定

- 强不可变性
- 静态类型系统
- 面向约束

工程化

- 多语言 SDK
- 内置函数+系统库
- YAML/JSON 亲和
- 包管理工具
- Plugin

KCL vs other DSLs

	KCL	HCL	CUE
建模能力	通过 KCL Schema 进行建模，通过语言级别的工程和部分面向对象特性，可以实现较高的模型抽象	通过 Terraform Go Provider Schema 定义，在用户界面不直接感知，此外编写复杂的 object 和必选/可选字段定义时用户界面较为繁琐	通过 Struct 进行建模，无继承等特性，当模型定义之间无冲突时可以实现较高的抽象。由于 CUE 在运行时进行约束检查，在大规模建模场景可能存在性能瓶颈
约束能力	更丰富的 check 声明式约束语法，编写更加容易，对于配置字段组合约束编写更加简单（比 CUE 多了 if guard 组合约束，all/any/map 等集合约束编写更容易）	通过 Variable 的 condition 字段对动态参数进行约束，Sentinel/Rego 等策略语言完成，语言本身的完整能力不能自闭环，且实现方式不统一	CUE 将类型和值合并到一个概念中，通过各种语法简化了约束的编写，比如不需要泛型和枚举，求和类型和空值合并都是一回事
扩展性	KCL 可以自定义配置分块编写方式和多种合并策略，KCL 同时支持幂等和非幂等的合并策略,可以满足复杂的多租户、多环境配置场景需求	Terraform HCL 通过分文件进行 Override, 模式比较固定，能力受限	支持语言内部配置合并，CUE 的配置合并是完全幂等的，对于满足复杂的多租户、多环境配置场景的覆盖需求可能无法满足
语言化编写能力	复杂的结构定义、循环、条件约束场景编写简单	编写复杂的对象定义和必选/可选字段定义时用户界面较为繁琐	对于复杂的循环、条件约束场景编写复杂，对于需要进行配置精确修改的编写场景较为繁琐

AppConfiguration 模型

Workload

应用工作负载描述，支持多种形态的应用负载，包括长时间运行的后端服务、一次性任务、有状态服务等

nginx

容器

Accessories

描述工作负载依赖的基础设施能力，例如数据库、负载均衡、对象存储等，并自动注入访问基础设施依赖的敏感信息



PolicySets

定义应用程序交付过程中需要遵循的安全性、合规性和技术风险策略，以最大限度地降低配置错误带来的风险。



mandatory



advisory



warning



approval

```
10 helloworld: ac.AppConfiguration {
11   # 工作负载配置
12   workload: s.Service {
13     containers: {
14       nginx: c.Container {
15         image: "nginx:v1"
16         command: ["/bin/sh", "-c", "echo hi"]
17         env: { "key": "value" }
18         resources: c.ComputeResource {
19           "cpu": "4"
20           "memory": "8192Mi"
21         }
22       }
23     }
24     replicas: 2
25   }
26   # 依赖的基础设施能力配置
27   accessories: {
28     "helloworld-db": db.OceanBase {
29       shardingSize: 100
30     }
31     "objStore": oss.Bucket {
32       presetQuota: 100
33     }
34   }
35   pipeline: {
36     "deploy": ops.DeployStrategy {
37       manualApprove: True
38       rollbackIfFailed: True
39     }
40   }
41   policysets: [
42     cp.ResourceCheck {
43       kind: "advisory"
44     }
45   ]
46   dependency: {
47     dependentApps: ["apiserver"]
48   }
49 }
```

- 面向开发者的平台界面，屏蔽底层复杂性，降低开发者认知
- 覆盖应用全生命周期，一份配置即可完整描述应用

Pipeline

声明应用交付过程中可定制的步骤，例如预检、审批、部署和后检。

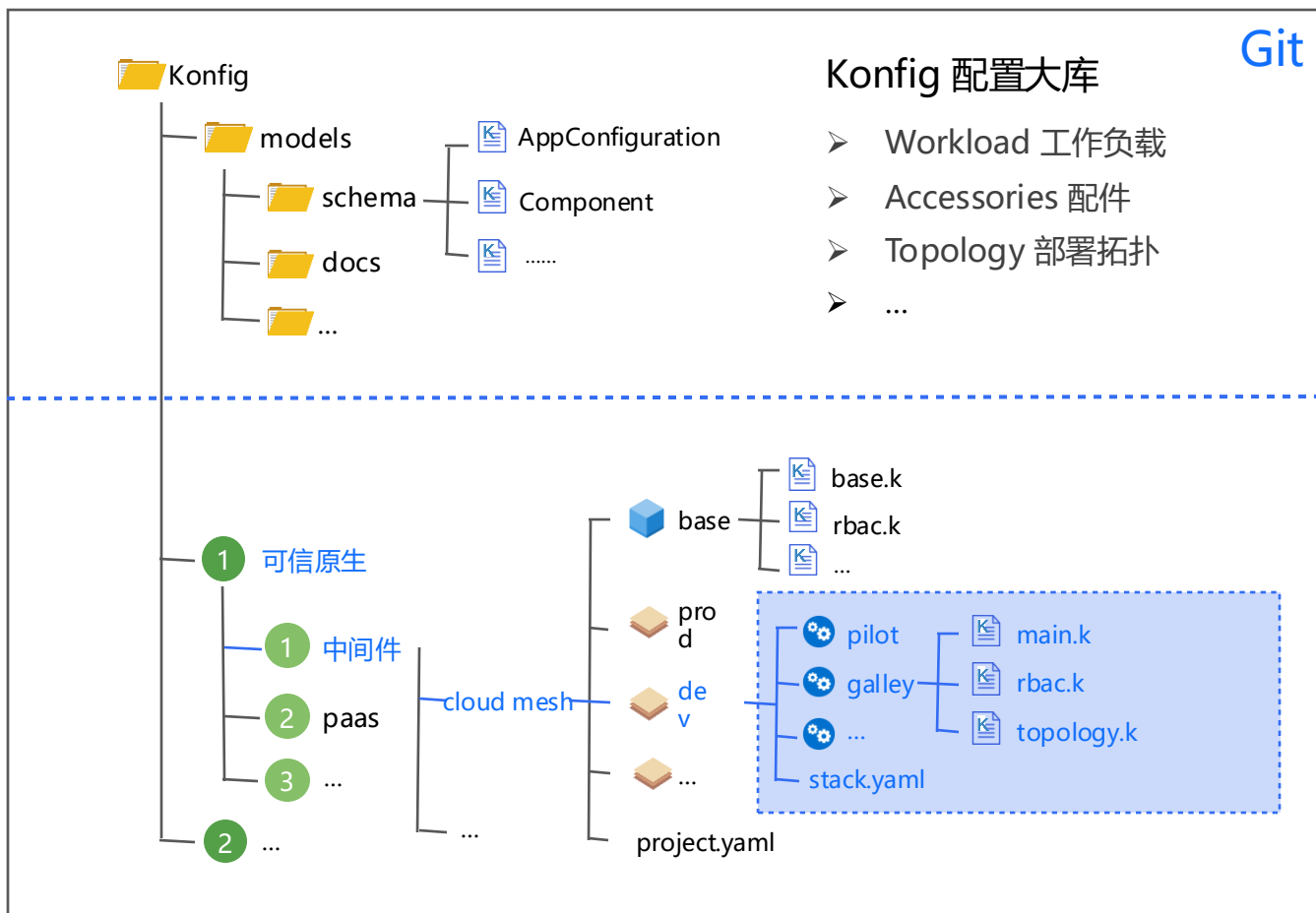


Dependency

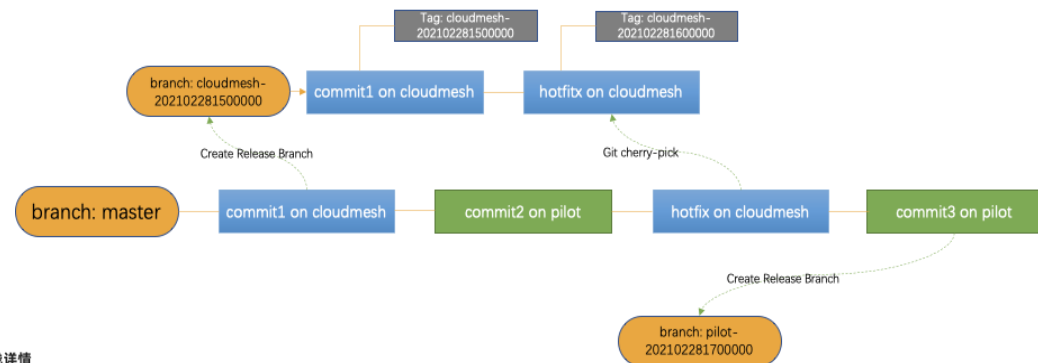
声明跨应用的依赖关系。



Konfig 大库



- 单一大库收拢蚂蚁所有应用交付配置
- 主干开发，分支发布
- 环境维度权限控制、对维度 CI 配套



流水线详情

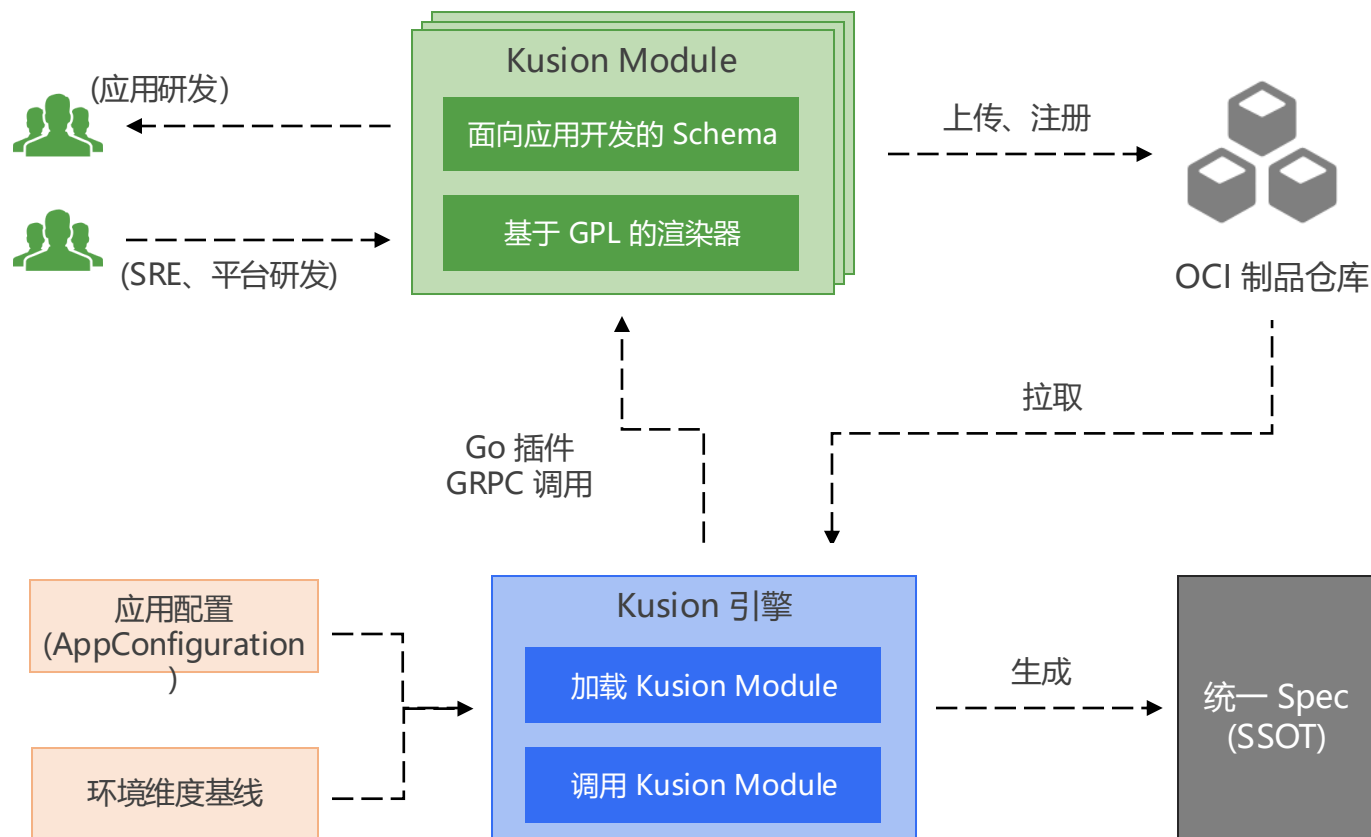
蚂蚁代码服务于 29分钟前 向 master PUSH 代码 #361762015 成功

aciyml | 00:04:30 | Git事件 Push | master | b77d5917 | 文件变更

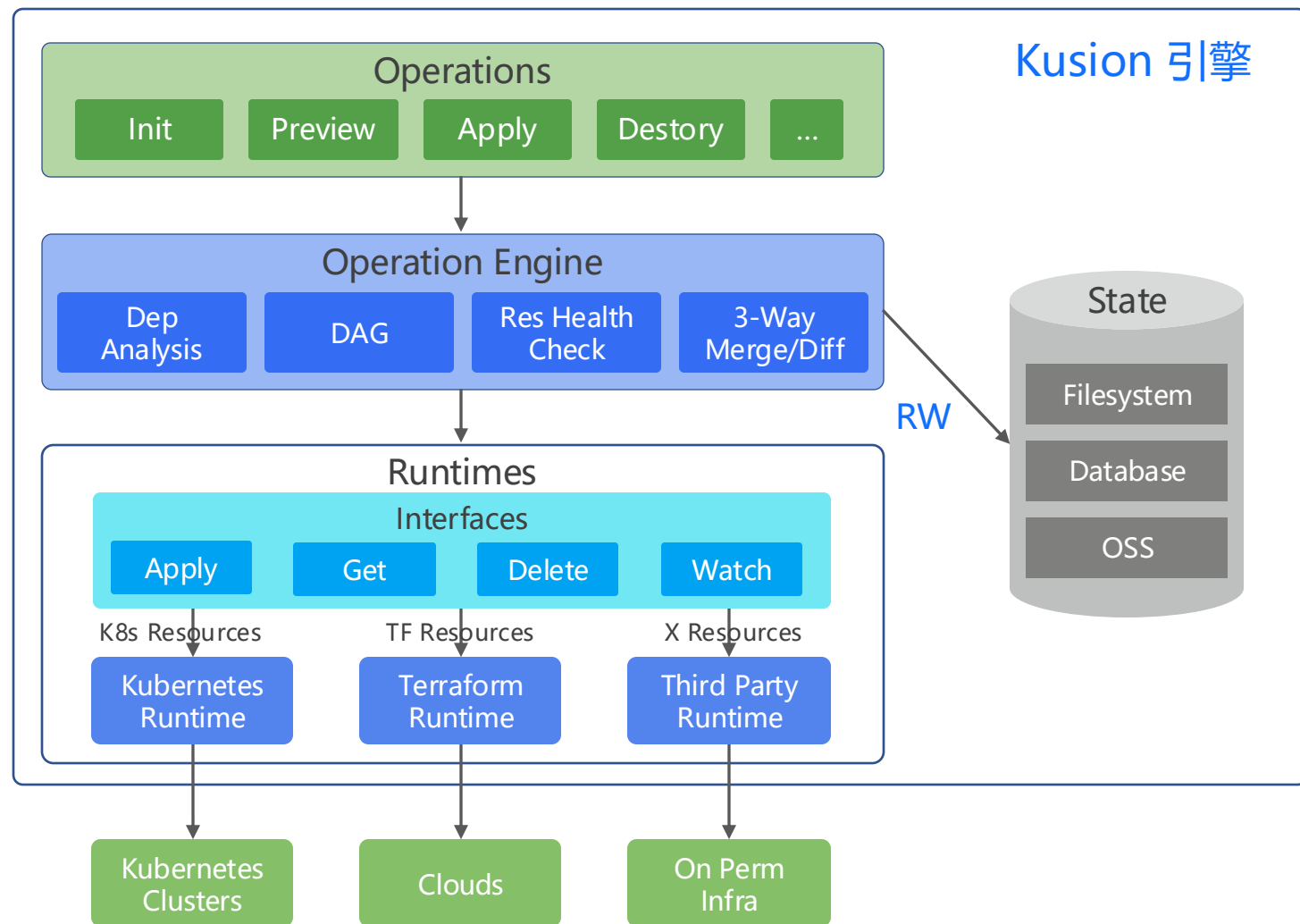


Kusion Module

- Kusion Module 是对底层基础设施服务、发布运维等平台能力的封装抽象，定义遵循单一职责原则，而非抽象整个云服务
- Kusion Module 由两部分组成：面向应用开发者的基于 KCL 的 Schema 定义，以及基于通用编程语言的 Generator 程序，负责实现对应的配置渲染、生成逻辑
- Kusion Module 的分发依赖 OCI 制品仓库，Kusion Engine 访问 OCI 中心拉取对应的 Kusion Module 制品，并以 Go 插件形式调用



Kusion 引擎



Kusion: 面向开发者的云原生接入工具，提供配套工具能力支持

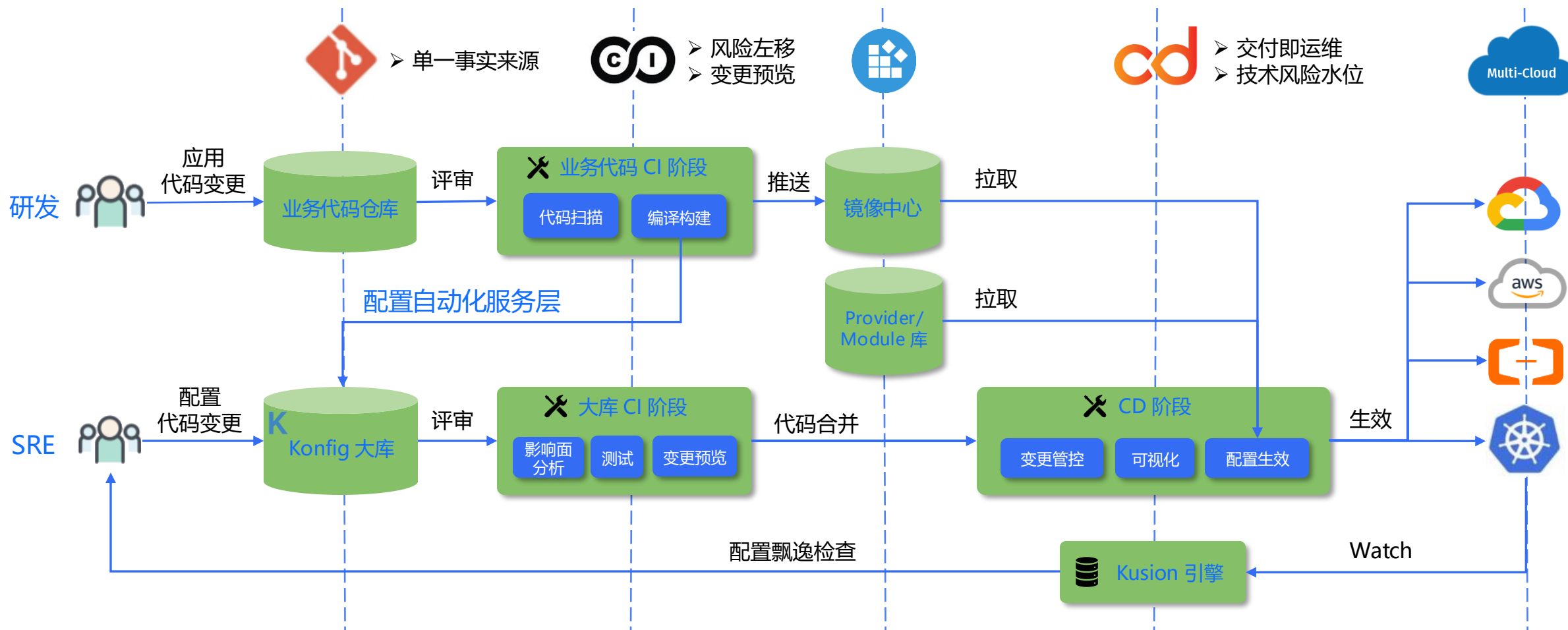
Engine: 提供终端命令依赖的资源管理、DAG编排、三路对比等核心功能

Runtimes: 抽象统一的基础设施接入层，可扩展方式实现异构基础设施一致交互

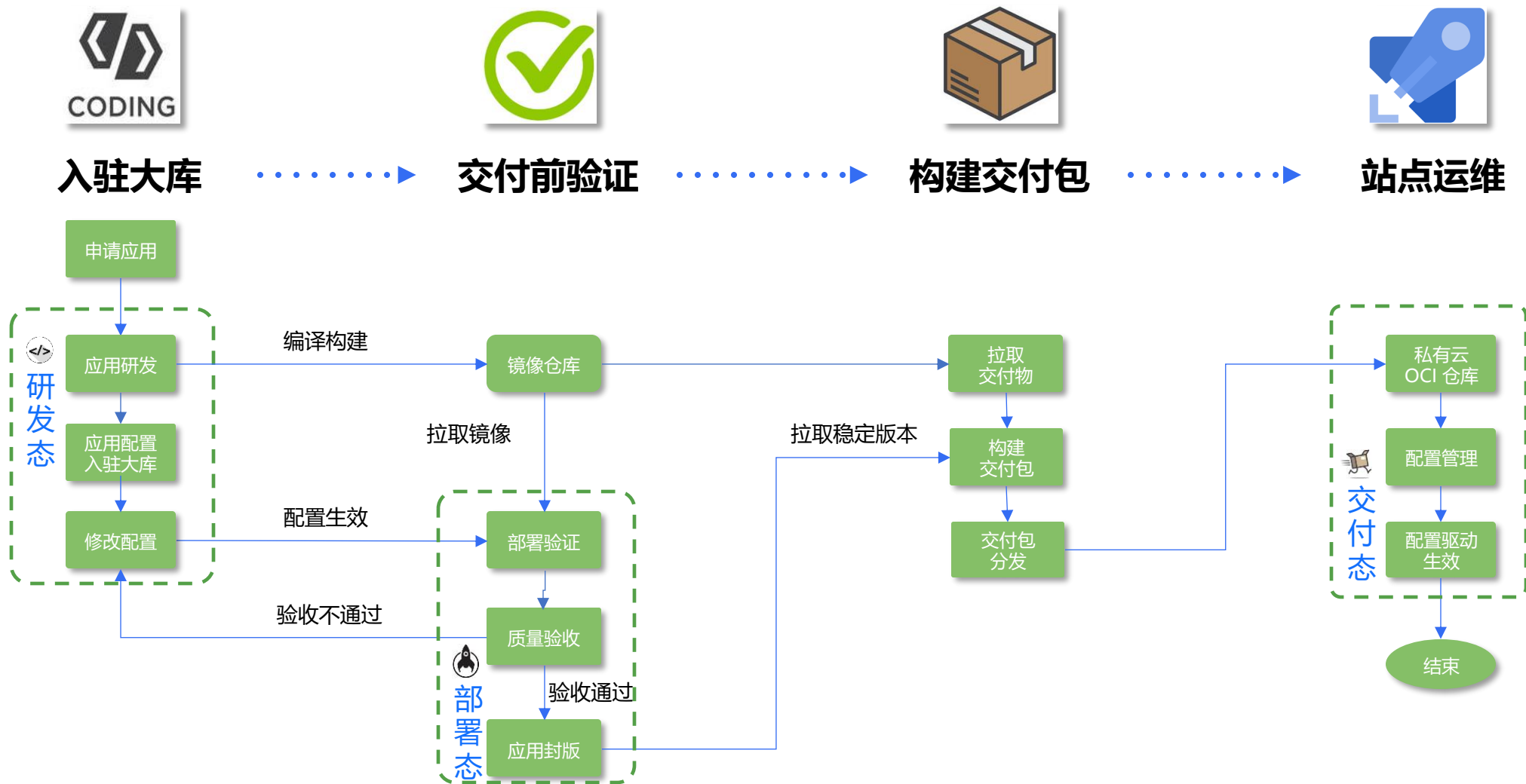
State: 真实世界资源在引擎中的映射，Kusion管理基础架构的必要数据

03 蚂蚁落地实践

配置驱动的应用变更



配置驱动的产品版本化交付





蚂蚁落地数据

100+

Kusion Modules

3000+

业务应用

1500+

蚂蚁开发者

470000+

Konfig 大库提交

1200+/天

CI 流水线

200M+

KCL Code

04 总结与展望

篇章页小标题（没有小标题可删除）



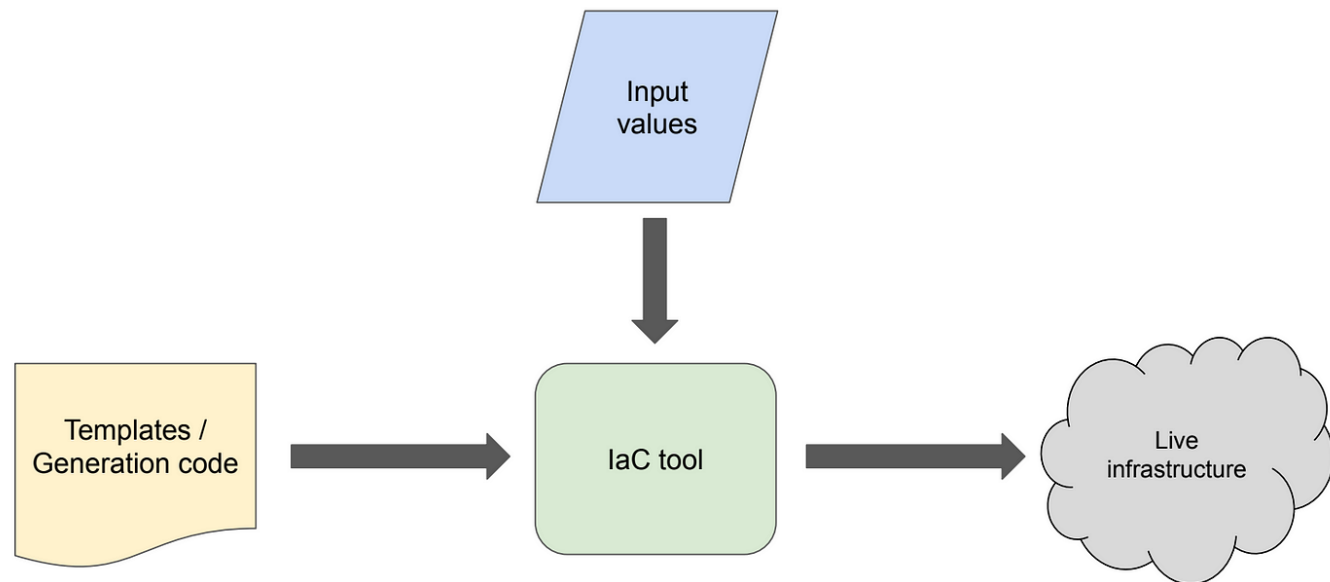
配置管理的复杂性

SOME OPEN, HARD PROBLEMS

Even with years of container-management experience, we feel there are a number of problems that we still don't have good answers for. This section describes a couple of particularly knotty ones, in the hope of fostering discussion and solutions.

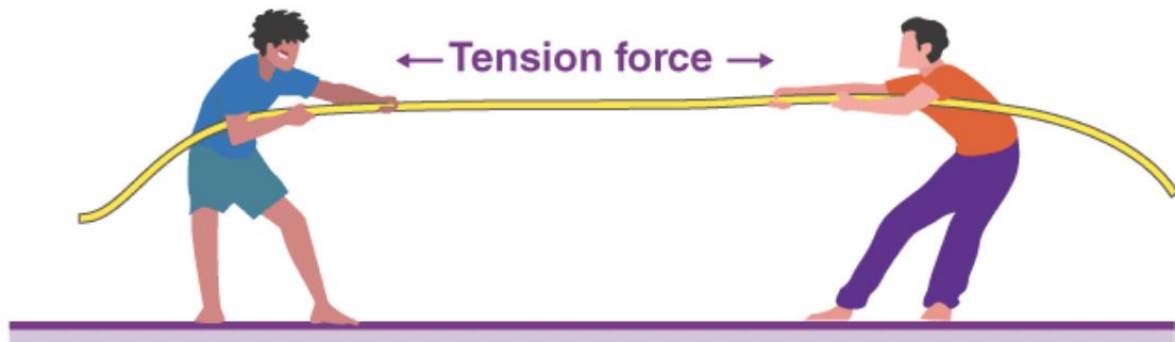
Configuration

Of all the problems we have confronted, the ones over which the most brainpower, ink, and code have been spilled are related to managing *configurations*—the set of values supplied to applications, rather than hard-coded into them.



透过现象看本质，不同产品、工具解决问题的思路本质是一样的，都面临根本性挑战

配置即代码的根本挑战



挑战一：灵活性与简单性之间的矛盾

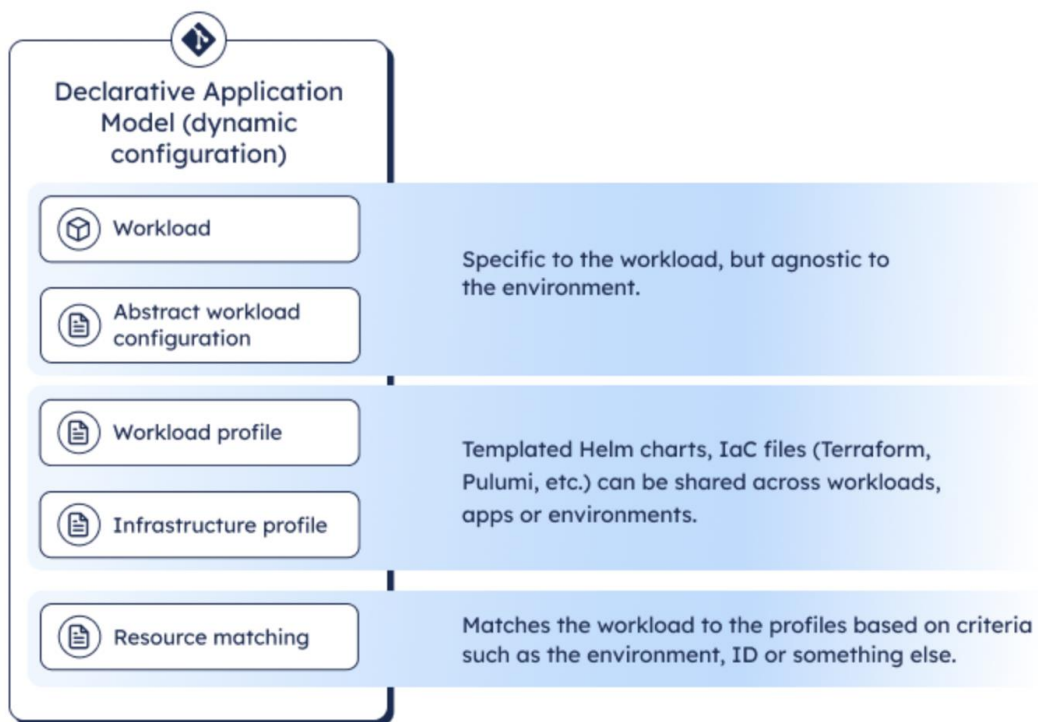
挑战二：配置代码独家驱动导致的不对称



~~控制台操作~~

配置代码驱动 ✓

● 动态配置管理 for IDP



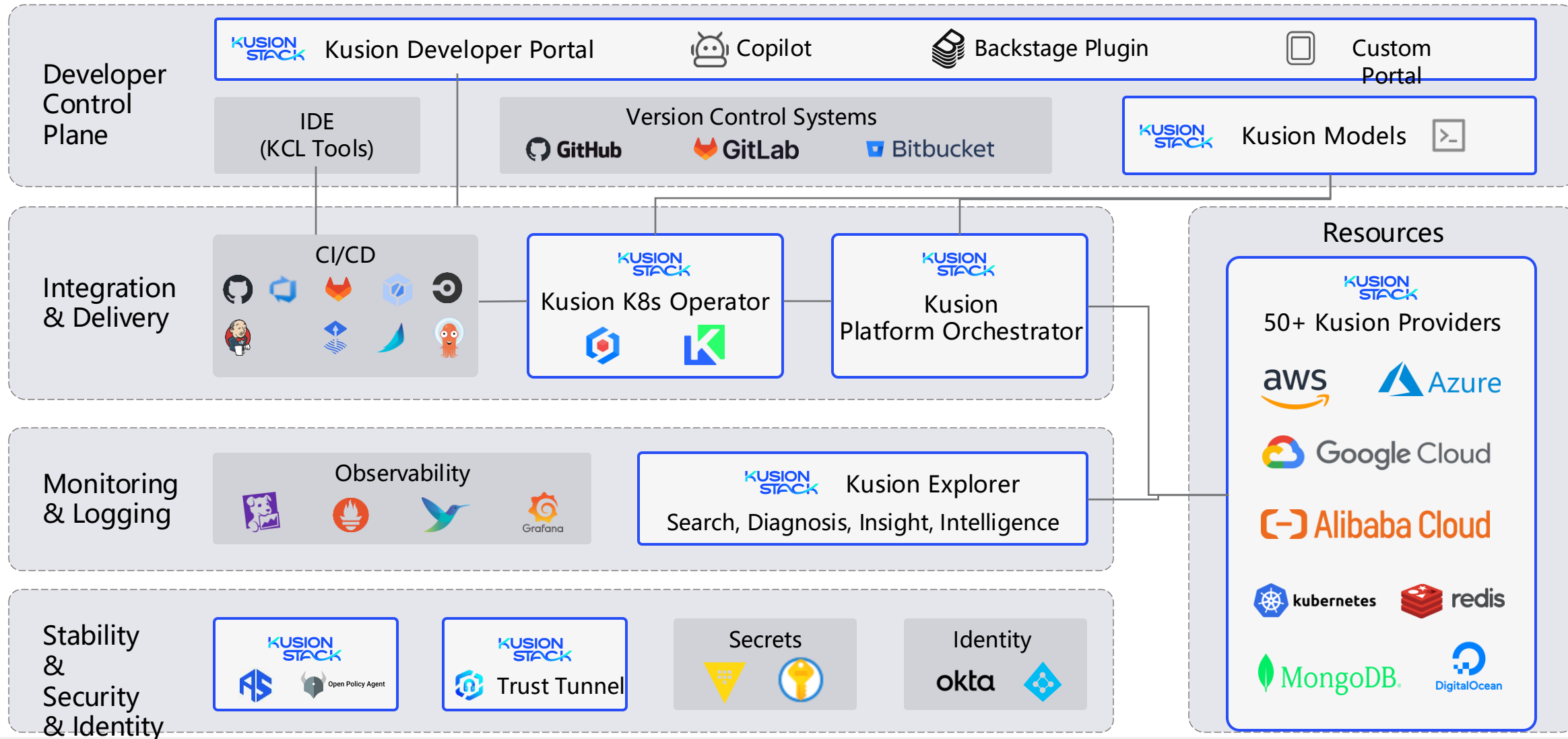
“Dynamic Configuration Management (DCM) is a methodology used to structure the configuration of compute workloads. Developers create workload specifications, describing everything their workloads need to run successfully. The specification is then used to dynamically create the configuration, to deploy the workload in a specific environment. With DCM, developers do not need to define or maintain any environment-specific configuration for their workloads.”



如果你计划实施平台工程战略，请务必确保你的平台支持动态配置管理

Dynamic Internal Developer Platform enables standardization across the organization

KusionStack for Platform Team



欢迎关注我们，欢迎 Star

- Website
 - <https://kusionstack.io/>
- Github
 - <https://github.com/KusionStack>
 - <https://github.com/kcl-lang>
- Slack
 - <https://kusionstack.slack.com>
- Twitter
 - [@KusionStack](https://twitter.com/KusionStack)





THANKS

大模型正在重新定义软件

Large Language Model Is Redefining
The Software

